

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CAMPUS ARARANGUÁ
GRADUAÇÃO EM TECNOLOGIA DA INFORMAÇÃO E COMUNICAÇÃO

Erick Navarro Vicentin & Liniker Albarello

RELATÓRIO DE MANUTENÇÃO E EVOLUÇÃO DE SOFTWARE

Araranguá
2026

1. IDENTIFICAÇÃO E CARACTERIZAÇÃO DO PROJETO **3**
2. BASELINE **3**
3. ANÁLISE DE MANUTENIBILIDADE E REFATORAÇÕES REALIZADAS **4**
Problema 1: 4
Problema 2: 4
Problema 3: 5
4. SOLICITAÇÃO DE MANUTENÇÃO PERFECTIVA **6**
5. ANÁLISE DE IMPACTO **6**
6. VERIFICAÇÃO E TESTES **7**
7. REFLEXÃO FINAL **8**

Projeto Open Source Analisado: pygame-chess

Repositório Original: https://github.com/mandrelbrotset/pygame-chess

Repositório do Grupo: https://github.com/linkmds/pygame-chess.git

1. IDENTIFICAÇÃO E CARACTERIZAÇÃO DO PROJETO

URL do Projeto Original: https://github.com/mandrelbrotset/pygame-chess

Objetivo do Software: Implementação gráfica do jogo de xadrez em duas dimensões para dois jogadores locais em turnos alternados.

Tecnologias Utilizadas: Python 3, biblioteca Pygame.

Justificativa da Escolha: O projeto apresenta um escopo bem delimitado e sem dependências de infraestrutura complexas (APIs externas ou bancos de dados). O código possui alto grau de acoplamento e oportunidade clara de análise de manutenibilidade e refatoração.

2. BASELINE

• **Commit da Baseline:** fc4f019b21903a427f8f04aa1d57d8c9a1b40c78 (Branch principal: master)

• **Build / Sintaxe:** Sucesso (python main.py executa normalmente sem erros de compilação/interpretação).

• **Quantidade de Testes Existentes na Baseline:** 0 (o projeto original de terceiros não continha testes automatizados).

• **Resultado dos Testes:** N/A.

• **Execução da Aplicação:** Sucesso.

3. ANÁLISE DE MANUTENIBILIDADE E REFATORAÇÕES REALIZADAS

Problema 1:

Estruturas de Dados Frágeis e Acoplamento por Listas/Tuplas (Magic Values)

Localização: Classe Chess em chess.py (métodos reset, move_piece, draw_pieces).

Evidência no Código Original:

```python
self.piece_location[chr(i)][a] = ["", False, [x, y]]
# Acessos por índices mágicos:
piece_name = value[0]
is_selected = value[1]
x_y_coord = value[2]
```

Consequência: Acessos indexados tornavam o código frágil. Qualquer modificação na ordem dos elementos da lista quebraria o funcionamento em múltiplos métodos da classe.

Técnica de Refatoração Utilizada: Substituição de Estruturas Genéricas por Objetos/Dataclasses (Encapsulamento de Dados).

Justificativa: Substituir listas/tuplas genéricas por objetos tipados com atributos nomeados aumenta a legibilidade, reduz bugs silenciosos e facilita alterações futuras.

Commit correspondente no Git: "refactor: refatoracao de manutenibilidade e estruturas do tabuleiro" (Branch: manutencao).

Problema 2:

Violação do Princípio da Responsabilidade Única (God Class / Métodos Gigantes)

Localização: Classe Chess em chess.py.

Evidência no Código Original: A classe Chess centralizava o gerenciamento de turnos, o cálculo das regras de movimento de todas as peças (possible_moves,

diagonal_moves, linear_moves), o tratamento de eventos e a renderização gráfica via Pygame (draw_pieces).

Consequência: Métodos extensos com lógica relacional complexa tornavam os testes unitários inviáveis e aumentavam o risco de efeitos colaterais.

Técnica de Refatoração Utilizada: Extração de Classes e Separação de Responsabilidades (SRP).

Justificativa: Delegar a lógica do tabuleiro e a validação de regras de peças para classes especialistas isola as regras de negócio da camada de interface/renderização.

Commit correspondente no Git: "refactor: refatoracao de manutenibilidade e estruturas do tabuleiro" (Branch: manutencao)

Problema 3:

Duplicação de Código Extrema e Validação Baseada em Strings Frágeis

Localização: Métodos possible_moves, move_piece e draw_pieces em chess.py.

Evidência no Código Original:

```python
if value[0][:5] == "black": ...
elif value[0][:5] == "white": ...
if piece_name[6:] == "bishop": ...
```

Consequência: O uso de fatiamento de strings e palavras-chave (*magic strings*) gerava alto risco de erros de digitação (*typos*), além de forçar a duplicação de blocos inteiros para tratar cores diferentes.

Técnica de Refatoração Utilizada: Substituição de Strings por Enumerações (enum.Enum) e Polimorfismo.

Justificativa: Garantir verificação em tempo de execução/desenvolvimento via enums de Cor (WHITE, BLACK) e Tipo de Peça, eliminando a manipulação de substrings.

Commit correspondente no Git: "refactor: refatoracao de manutenibilidade e estruturas do tabuleiro" (Branch: manutencao)

4. SOLICITAÇÃO DE MANUTENÇÃO PERFECTIVA

Funcionalidade Proposta: Promoção do Peão (*Pawn Promotion*).

Descrição da Necessidade: No xadrez tradicional, quando um peão atinge a última fileira do tabuleiro adversário (linha 0 para brancas e linha 7 para pretas), ele deve ser promovido a uma peça mais forte (Rainha). O código original não tratava esse evento, mantendo o peão com limitações de movimento no final do tabuleiro.

Comportamento Esperado: Ao mover um peão branco para a linha de índice 0 ou um peão preto para a linha de índice 7, a peça é convertida automaticamente em uma Rainha (white_queen ou black_queen).

Regras / Restrições:

Peão branco (white_pawn) movido para y == 0 → transforma-se em white_queen.

Peão preto (black_pawn) movido para y == 7 → transforma-se em black_queen.

Critérios de Aceitação: A representação interna do tabuleiro e o sprite exibido na tela devem ser atualizados imediatamente para a Rainha da respectiva cor, e a peça deve passar a se movimentar como Rainha no turno seguinte.

5. ANÁLISE DE IMPACTO

Impacto Previsto:

Elemento afetado: Método de execução/validação de jogadas na classe responsável pelo tabuleiro/regras (validate_move em chess.py).

Interseção: Nenhuma alteração direta nos loops de eventos de mouse do Pygame ou no arquivo game.py.

Impacto Efetivamente Observado:

O impacto manteve-se restrito ao método de movimentação do peão/tabuleiro (validate_move).

A renderização (draw_pieces) respondeu automaticamente à mudança do identificador/enum da peça sem necessidade de modificações adicionais.

6. VERIFICAÇÃO E TESTES

Para garantir a preservação dos comportamentos existentes e validar a nova funcionalidade, foi desenvolvida uma suíte de testes automatizados utilizando a biblioteca pytest (com suporte a mocks da biblioteca unittest.mock para isolar a interface gráfica do Pygame). O arquivo de testes (test_chess.py) encontra-se versionado na branch manutencao.

Testes Implementados (test_chess.py):

test_initial_board_setup: Confirma a correta disposição inicial das peças no tabuleiro.

test_pawn_moves_and_promotion: Valida o avanço do peão e comprova que a promoção de Peão para Dama (white_queen) ocorre com sucesso ao atingir a fileira de destino (y == 0 ou des_row_no == 8).

test_knight_moves, test_king_moves, test_bishop_diagonal_moves, test_rook_linear_moves, test_queen_combined_moves: Verificam a integridade do cálculo de movimentos de todas as peças.

Resultados Obtidos:

Quantidade de testes executados: 7 testes.

Taxa de sucesso: 100% de aprovação (0 falhas, 0 erros).

Comparação com a Baseline: A baseline original não continha testes automatizados (0 testes). A versão modificada estabelece uma suíte de testes de regressão que valida tanto as refatorações quanto a manutenção perfectiva.

7. REFLEXÃO FINAL

Principais Dificuldades: A compreensão da estrutura legada, especialmente o uso de dicionários aninhados com chaves numéricas e alfabéticas misturadas (self.piece_location[chr(i)][a]) sem tipagem e sem testes existentes para orientar o comportamento.

Aprendizados: Compreensão prática de como maus hábitos de codificação (como *magic values*, fatiamento de strings e *God Class*) aumentam exponencialmente o esforço de manutenção. A aplicação de refatoração estruturada aliada a testes garante evoluções seguras em sistemas de terceiros.